Week 8, Lecture 2: Optimization

Instructor notes. Part 1: prep reading. Part 2: podium cheat sheet

How to prepare (for students)

- **Big picture.** Since Week 6 we've found maximum likelihood estimates by handing a negative log-likelihood to `optim()` and trusting what came back. Today we open the black box: how do optimizers search for the lowest point of an NLL surface, and why do they sometimes stop in the wrong place?
- **Bring:** your take-home WOD on Newton's method. We start by going over it.
- **Read:** Bolker 7.1-7.2 (printed p. 294-310). Skim 7.4.4 on multiple minima (p. 323-326).
- **Review:** derivatives and the Hessian from Lab 6 (the curvature of the NLL), and the Gamma distribution (Bolker 4.5.2.3, p. 172-174). Today's example fits a Gamma distribution to virus titers.

Overview (instructor only)

This file has two parts, both instructor-only. **Part 1** is read beforehand. **Part 2** (after the page break, large print) is the podium copy.

Beat	Time
Take-home WOD debrief (discussion)	~8 min
Lecture, with three short discussion pauses	~40 min
Separate discussion period	none

New workflow framing. Section 1 introduces the revised scientific workflow (PROBLEM → THEORIES → MODELS → FIT → COMPARE → REVISE, with a study-design side arrow from MODELS to DATA). Draw it in full once today; later lectures can use the one-line version. To make room, section 1 grows to ~5 min and Nelder-Mead drops to ~10 min.

Pacing. This is a full period by design. Core: sections 0-3 and the start of section 4 (the rules and the acceptance probability). Whatever of section 4 doesn't fit opens Week 9 Monday, where it leads straight into MCMC (same acceptance rule). The "if time" section at the end (reparameterizing the Gamma, a fitting checklist) can go to Week 9 or Lab 7. The three discussion pauses are short (~2 min each); if running behind, ask the question, take one answer, and give the landing point yourself.

Source material. Bolker 7.1-7.2 (direct search, Newton's method, BFGS, Nelder-Mead, simulated annealing), 7.4.4 (multiple minima; the Vonesh and Bolker story); legacy `LECTURE5.Rmd` ("Optimization!": myxomatosis example, brute force, Newton with `pracma`, hand-coded simplex and simulated annealing). The code below is rewritten from the legacy version: no extra packages beyond `emdbook`, and one set of helper functions shared across sections.

Example choice. The myxomatosis titers (grade 1 virus, 27 rabbits) with a Gamma distribution, shape and scale parameters. This is Bolker's running example in 7.2 (his Figures 7.5 and 7.6), so the reading and the lecture line up. It was mentioned only in passing in Week 6. Two features make it a good teaching surface: the NLL is undefined for negative parameters, and shape and scale are strongly correlated, so the minimum sits in a long, narrow, curved valley. The pikas carry over from the take-home for the one-dimensional case.

Where this sits in the week

- **Monday:** finished the one-parameter Bayesian material (germination example); handed out the Newton's method take-home.
- **Tuesday:** group meetings on final project proposals (no lab).
- **Today:** how optimizers work, and how they fail.
- **Coming up:** Week 9 Monday picks up simulated annealing and turns it into MCMC; Lab 7 on Bayesian inference.

Part 1: Lecture content

```
library(emdbook)
titer <- subset(MyxoTiter_sum, grade == 1)$titer

# gamma NLL; Inf outside the allowed range so optimizers can't use it
nll_gamma <- function(p) {
  if (any(p <= 0)) return(Inf)
  -sum(dgamma(titer, shape = p[1], scale = p[2], log = TRUE))
}

# central differences: f'(x) ~ [f(x + delta) - f(x - delta)] / (2 delta)
grad_fd <- function(f, p, delta = 1e-4 * abs(p)) {
  sapply(seq_along(p), function(i) {
    e <- replace(numeric(length(p)), i, delta[i])
    (f(p + e) - f(p - e)) / (2 * delta[i])
  })
}

# matrix of second derivatives (the Hessian), by finite differences
hess_fd <- function(f, p, delta = 1e-3 * abs(p)) {
```

```
  k <- length(p)
  H <- matrix(0, k, k)
  for (i in seq_len(k)) {
    for (j in seq_len(k)) {
      ei <- replace(numeric(k), i, delta[i])
      ej <- replace(numeric(k), j, delta[j])
      H[i, j] <- (f(p + ei + ej) - f(p + ei - ej) - f(p - ei + ej) +
                    f(p - ei - ej)) / (4 * delta[i] * delta[j])
    }
  }
  H
}

shape_vec <- seq(10, 90, length.out = 120)
scale_vec <- seq(0.03, 0.32, length.out = 120)
surf <- outer(shape_vec, scale_vec, Vectorize(function(a, s) nll_gamma(c(a, s))))
nll_min <- min(surf)

# contour plot of NLL minus its minimum over the shape-scale grid
plot_surface <- function(...) {
  contour(shape_vec, scale_vec, surf - nll_min,
          levels = c(1, 2, 5, 10, 20, 50, 100, 200, 500, 1000),
          xlab = "shape", ylab = "scale", col = "grey60", labcex = 0.6, ...)
}
f2 <- function(v) formatC(v, format = "f", digits = 2)
f3 <- function(v) formatC(v, format = "f", digits = 3)
```

0. Take-home debrief (~8 min, discussion)

Run this as a discussion, not a solution walkthrough. Key in `wod/answer_keys/WOD_Week8_Lecture1_Newto`

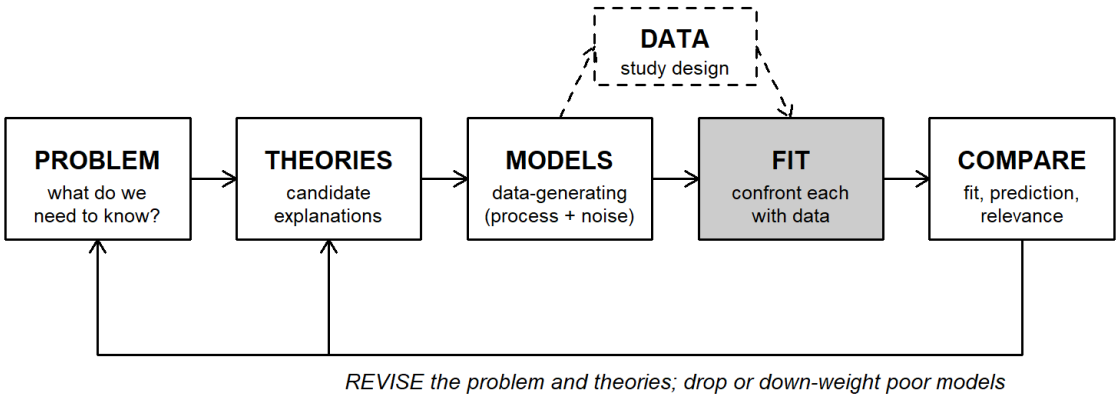
- **3a (deriving the update).** Ask a student to put their λ^* on the board: $\lambda^* = \lambda - \mathrm{NLL}'/\mathrm{NLL}''$, from setting the straight-line slope $\mathrm{NLL}'(\lambda) + \mathrm{NLL}''(\lambda)(\lambda^* - \lambda)$ to zero. Note that a straight-line slope means a parabola-shaped NLL, so each step jumps to the bottom of a parabola. Dividing by the curvature sets the step size.
- **3b (Newton from $\lambda = 2$).** Ask for the guesses: 2, 2.667, 2.963, 3.000. Ask how fast the error shrank: 1, 0.33, 0.037, 0.0005. Each error is roughly the square of the last. That speed near the answer is the appeal of derivative-based methods.
- **3c (start at $\lambda = 7$).** Ask who got a negative number (-2.33) and why. Far from the MLE the slope NLL' levels off, so the straight line drawn at 7 is shallow and doesn't cross zero until past $\lambda = 0$. Collect fixes on the board: shorter steps (halve until the NLL improves), better starting values, bounds, fitting on the log scale. Newton on $\theta = \log \lambda$ from 7 goes 3.95, 3.11, 3.002: no overshoot.
- **2a, 2b, and 3d (finite differences).** A forward difference with step Δ is really

the slope at the chord's midpoint (2a: the forward estimate at 2 matches the exact slope at 2.25). Taking that idea twice gives the second-difference formula, which is how `optim(..., hessian = TRUE)` got the Hessian in Lab 6. A Newton step built from the forward slope (2.44) heads the right way but falls short of the exact step (2.67); a smaller Δ (`optim()` uses 0.001) or a centered slope (2.69) fixes it. An optimizer doesn't need calculus.

Landing point: *every fix students proposed is built into a real optimizer.* Today: what those optimizers do in more than one dimension.

1. Big picture: the workflow, and where today sits (~5 min)

New framing, introduced today. Earlier lectures opened with a four-step loop (THEORY → MODEL → DATA → COMPARE). From today on, use this fuller version. Draw it on the board once in full; later lectures can use the one-line form and point at a box.



- **PROBLEM** first: what do we need to know? Writing it down first is what lets us later ask whether a model is *relevant*, not just whether it fits.
- **THEORIES:** candidate explanations, more than one where possible (multiple working hypotheses).
- **MODELS:** each theory becomes a data-generating model, a deterministic function plus a probability distribution (Weeks 3-5).
- **DATA (side arrow):** the models tell us what data to collect and how much (the Week 6 power analysis by simulation).
- **FIT:** confront each model with the data (likelihood, Bayes).
- **COMPARE:** does it fit, does it predict, does it answer the problem? Poor models are dropped (falsified) or down-weighted. The survivors are corroborated, not proven.
- **REVISE:** loop back to the problem and the theories, not just the parameters.

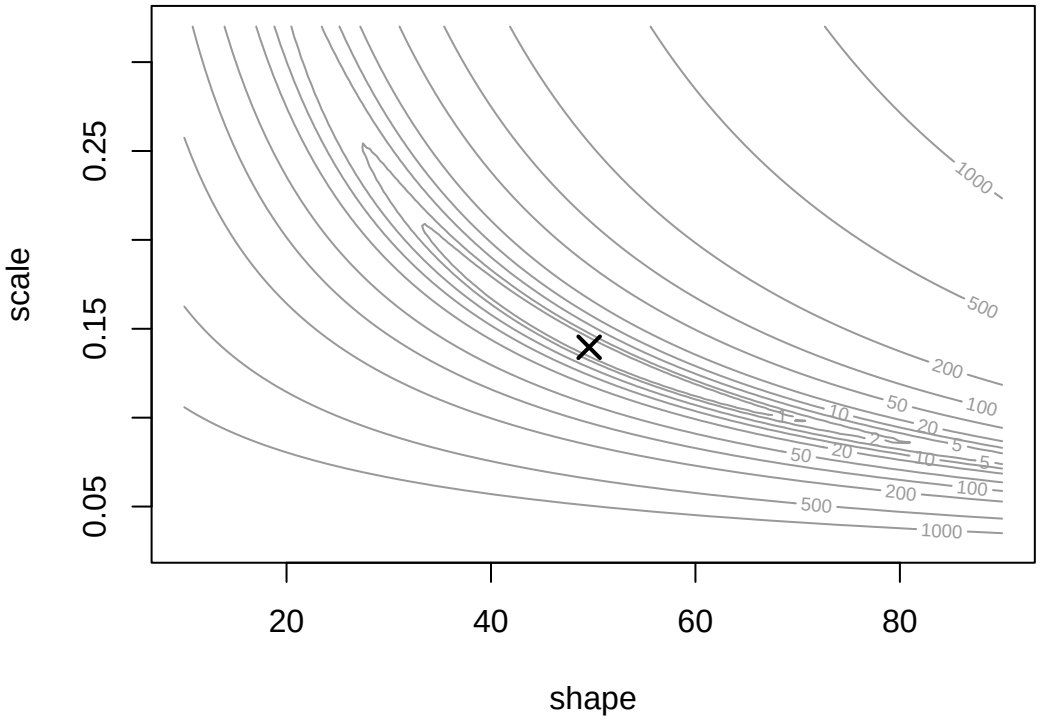
Today is the FIT box. Fitting means finding the lowest point of the NLL surface. Every method today is a strategy for searching that surface without evaluating it everywhere,

and every discussion pause today is about not trusting a fit blindly before moving on to COMPARE.

Sticking point. Students may hear "down-weight" as "pick the winner and call it true." Head that off: weighting ranks the models that survived; it says nothing about models nobody proposed.

- **Today's data.** Virus titers ($\log_{10}$ virus load) in 27 rabbits infected with the most virulent grade of myxomatosis (Bolker's example). Model: Gamma(shape, scale). Two parameters, so we can still draw the whole surface:

```
par(mar = c(4, 4, 0.5, 0.5))
plot_surface()
fit_nm <- optim(c(20, 0.05), nll_gamma)
points(fit_nm$par[1], fit_nm$par[2], pch = 4, cex = 1.5, lwd = 2)
```



- **Brute force works here** (Week 6 grids): this picture took 14,400 NLL evaluations. Bolker's coarser grid (shape 10 to 100 by 1, scale 0.01 to 0.3 by 0.01) takes 2,730.
- **It doesn't scale.** At 30 values per parameter, a 10-parameter model needs $30^{10} \approx 6 \times 10^{14}$ evaluations: at 1 ms each, about 19,000 years.
- **The MLE.** Shape 49.60, scale 0.140, NLL 37.667.
- Note the shape of the valley: long, narrow, and curved. That shape causes trouble for every method today.

2. Newton's method in more than one dimension (~10 min)

From the take-home. One parameter: step $= -\text{NLL}'/\text{NLL}''$. Two or more parameters:

- the slope becomes the **gradient**, a vector of slopes, one per parameter;
- the curvature becomes the **Hessian**, a matrix of second derivatives (Lab 6);
- "divide by the curvature" becomes "multiply by the inverse of the Hessian": $\theta_{\text{new}} = \theta - H^{-1} g$. In R, `solve(H, g)`.

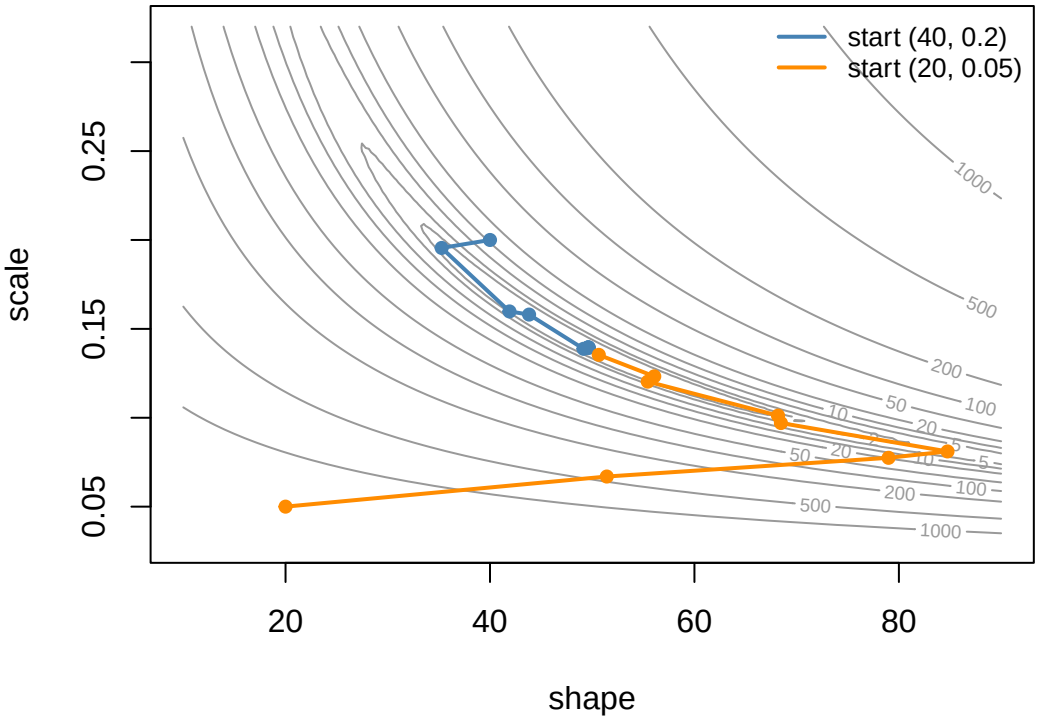
```
newton <- function(f, start, n_steps = 8) {
  path <- matrix(start, nrow = 1)
  p <- start
  for (i in seq_len(n_steps)) {
    p <- p - solve(hess_fd(f, p), grad_fd(f, p))
    path <- rbind(path, p, deparse.level = 0)
  }
  path
}
path_good <- newton(nll_gamma, c(40, 0.2))
path_bad <- newton(nll_gamma, c(20, 0.05))
round(cbind(path_good, nll = apply(path_good, 1, nll_gamma)), 3)
```

```
##                       nll
##  [1,] 40.000 0.200 48.703
##  [2,] 35.275 0.196 38.383
##  [3,] 41.907 0.160 38.486
##  [4,] 43.826 0.158 37.768
##  [5,] 49.152 0.139 37.817
##  [6,] 49.679 0.139 37.667
##  [7,] 49.612 0.140 37.667
##  [8,] 49.611 0.140 37.667
##  [9,] 49.611 0.140 37.667
```

```
round(cbind(path_bad, nll = apply(path_bad, 1, nll_gamma)), 3)
```

```
##                         nll
##  [1,] 20.000 0.050 2196.012
##  [2,] 51.428 0.067  471.654
##  [3,] 78.988 0.077   56.375
##  [4,] 84.756 0.081   40.084
##  [5,] 68.462 0.097   40.067
##  [6,] 68.179 0.101   38.445
##  [7,] 55.435 0.120   38.809
##  [8,] 56.072 0.123   37.775
##  [9,] 50.645 0.135   37.736
```

```
par(mar = c(4, 4, 0.5, 0.5))
plot_surface()
lines(path_good, type = "o", pch = 19, cex = 0.7, col = "steelblue", lwd = 2)
lines(path_bad, type = "o", pch = 19, cex = 0.7, col = "darkorange", lwd = 2)
legend("topright", c("start (40, 0.2)", "start (20, 0.05)"), lwd = 2,
       col = c("steelblue", "darkorange"), bty = "n", cex = 0.8)
```



- From (40, 0.2), Newton reaches the minimum in about six steps.
- From (20, 0.05), it lurches across the valley, the NLL goes *up* on one step, and after eight steps it still isn't done. Same lesson as 3c on the take-home: the straight-line slope (a parabola-shaped NLL) is a good guide near the minimum and a bad one far from it.
- **Why not always use Newton?** It needs the full Hessian at every step: for k parameters, about k^2 second derivatives, each costing several NLL evaluations by finite differences.
- **BFGS** (`method = "BFGS"`, a "quasi-Newton" method) computes only the gradient. It builds up an approximate Hessian from how the gradient changes from step to step. This is the workhorse for smooth surfaces (and `mle2()`'s default). If you can write the first derivatives as a function, pass them through `optim()`'s `gr` argument: no finite differences, so faster and more stable (Bolker p. 301; footnoted on the take-home).
- **One parameter?** Use `optimize()`, which brackets the minimum and narrows in. `optim()`'s default warns if you give it one parameter.

Pause for discussion (~2 min).

```r
fit_bfgs <- optim(c(20, 0.05), nll_gamma, method = "BFGS")
fit_bfgs_log <- optim(log(c(20, 0.05)), function(q) nll_gamma(exp(q)),
                      method = "BFGS")
cmp <- rbind(
  nelder_mead = c(fit_nm$par, fit_nm$value, fit_nm$counts[1], fit_nm$convergence),
  bfgs = c(fit_bfgs$par, fit_bfgs$value, fit_bfgs$counts[1], fit_bfgs$convergence),
  bfgs_log_scale = c(exp(fit_bfgs_log$par), fit_bfgs_log$value,
                     fit_bfgs_log$counts[1], fit_bfgs_log$convergence)
)
colnames(cmp) <- c("shape", "scale", "nll", "n_evals", "convergence")
round(cmp, 4)
```

```
##                  shape  scale     nll n_evals convergence
## nelder_mead    49.6026 0.1396 37.6671     147           0
## bfgs           48.6677 0.1424 37.6700     231           1
## bfgs_log_scale 49.6089 0.1396 37.6671      33           0
```

> Same data, same starting point. Nelder-Mead (`optim()`'s default) returns NLL 37.667 with `convergence = 0`. BFGS returns NLL 37.670 with `convergence = 1`, and estimates (48.67, 0.142) that look close to Nelder-Mead's. Is the BFGS fit done? What would you do?

Where to land: no. `convergence = 1` means BFGS hit its iteration limit (100) before it stopped improving. The NLL difference is tiny, but nothing guarantees it would be tiny on another problem. Always check `$convergence`. Fixes, from the take-home list: a better start, restarting from where it stopped, or a better-behaved scale. Fitting log(shape) and log(scale) instead converges in 33 evaluations (and can't wander into negative values). Watch for students treating "the numbers look close" as "converged."

3. Derivative-free: Nelder-Mead simplex (~10 min)

The idea. No slopes at all. Keep $k + 1$ points (a triangle for two parameters), and keep moving the worst one. Sketch on the board:

- **Reflect** the worst point through the midpoint of the other two.
- If that's the best point yet, **expand**: go twice as far.
- If the reflection is still bad, **contract**: try halfway back.
- If nothing works, **shrink** the whole triangle toward the best point.

The triangle tumbles downhill, stretches out along valleys, and shrinks around the minimum. `optim()` stops when the vertices' NLL values are all within a small tolerance.

```r
# hand-coded Nelder-Mead for two parameters; stores every simplex
nelder_mead <- function(f, start, step, n_iter = 80) {
  simplex <- rbind(start, start + c(step[1], 0), start + c(0, step[2]))
  vals <- apply(simplex, 1, f)
```

```
  history <- vector("list", n_iter)
  moves <- character(n_iter)
  for (it in seq_len(n_iter)) {
    o <- order(vals)
    simplex <- simplex[o, ]
    vals <- vals[o]
    history[[it]] <- simplex
    centroid <- colMeans(simplex[1:2, ])
    refl <- centroid + (centroid - simplex[3, ])
    f_r <- f(refl)
    if (f_r < vals[1]) {
      expd <- centroid + 2 * (centroid - simplex[3, ])
      f_e <- f(expd)
      if (f_e < f_r) {
        simplex[3, ] <- expd; vals[3] <- f_e; moves[it] <- "expand"
      } else {
        simplex[3, ] <- refl; vals[3] <- f_r; moves[it] <- "reflect"
      }
    } else if (f_r < vals[2]) {
      simplex[3, ] <- refl; vals[3] <- f_r; moves[it] <- "reflect"
    } else {
      contr <- centroid + 0.5 * (simplex[3, ] - centroid)
      f_c <- f(contr)
      if (f_c < vals[3]) {
        simplex[3, ] <- contr; vals[3] <- f_c; moves[it] <- "contract"
      } else {
        simplex[2:3, ] <- (simplex[2:3, ] + rbind(simplex[1, ], simplex[1, ])) / 2
        vals[2:3] <- apply(simplex[2:3, ], 1, f)
        moves[it] <- "shrink"
      }
    }
  }
  list(best = simplex[which.min(vals), ], value = min(vals),
       history = history, moves = moves)
}
nm <- nelder_mead(nll_gamma, c(20, 0.05), step = c(5, 0.02))
round(c(shape = nm$best[1], scale = nm$best[2], nll = nm$value), 4)
```

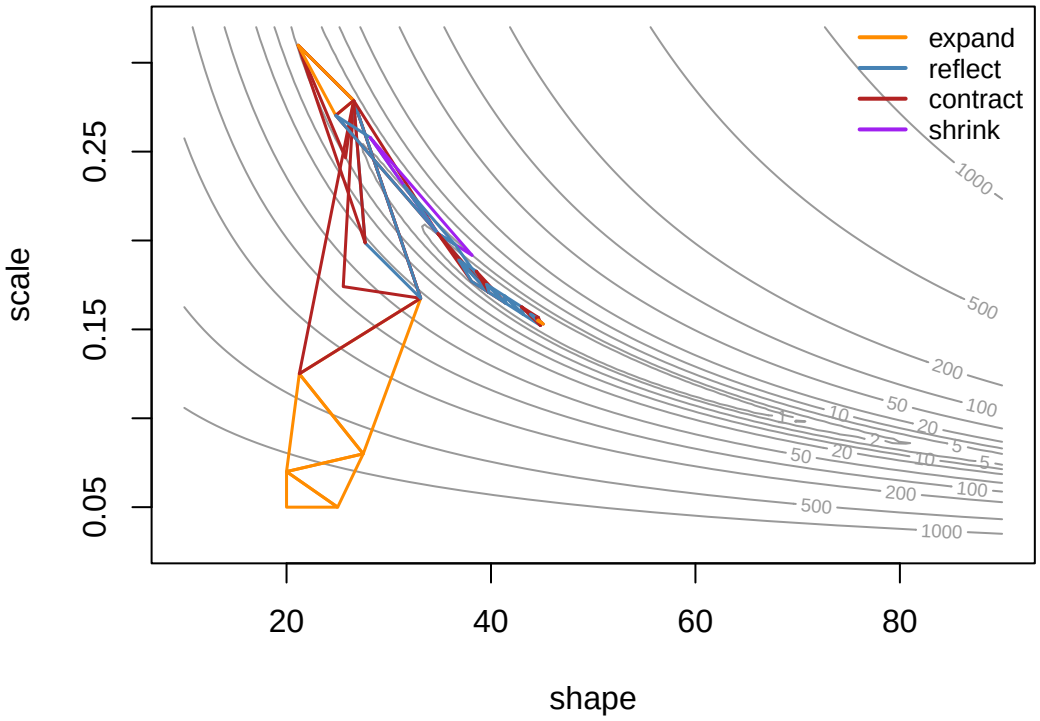
```
##   shape   scale     nll
## 49.6114  0.1396 37.6671
```

```
table(nm$moves)
```

```
##
## contract   expand  reflect   shrink
```

```
##       46        8       25        1
```

```
move_cols <- c(expand = "darkorange", reflect = "steelblue",
               contract = "firebrick", shrink = "purple")
par(mar = c(4, 4, 0.5, 0.5))
plot_surface()
for (i in 1:30) polygon(nm$history[[i]], border = move_cols[nm$moves[i]], lwd = 1.5)
legend("topright", names(move_cols), col = move_cols, lwd = 2, bty = "n", cex = 0.8)
```



- The first 30 triangles are shown. Big expanding steps across the contours, then contractions as it "turns the corner" into the valley, then reflections and contractions crawling along the valley floor (compare Bolker's Figure 7.5, p. 305).
- `optim()`'s version needed 147 NLL evaluations. BFGS on the log scale needed 33 (plus gradient evaluations). Nelder-Mead is slower but doesn't care whether the surface is smooth: bumps, kinks, and thresholds don't fool it the way they fool a slope-based method.

Pause for discussion (~2 min).

```
vc <- solve(optimHess(fit_nm$par, nll_gamma))
corr_ss <- cov2cor(vc)[1, 2]
prod(fit_nm$par)   # shape x scale = Gamma mean
```

```
## [1] 6.923736
```

```
mean(titer)
```

```
## [1] 6.924074
```

> Every method today crawled along the same long, curved valley. Using the Hessian at the MLE (Lab 6), the estimated correlation between shape and scale is -0.995. What does that say about what these data can and can't tell us? (Hint: the mean of a Gamma is shape × scale.)

Where to land: along the valley floor, shape goes up and scale goes down while their product, the mean titer (6.92), barely changes. The data are very informative about the mean and much less about how it splits between shape and scale. That's what makes the valley long and narrow, and it's why optimizers creep along it. Callback: in Lab 6 Act 1, centering the dose at 1.8 removed the same kind of correlation between a and b. Reparameterizing (fit mean and shape instead) is the fix here; it's in the "if time" section.

4. Stochastic global search: simulated annealing (start; ~8 min)

The problem. Everything so far goes downhill from the start. If the surface has more than one valley, a downhill method finds whichever valley it starts in. Sketch on the board: an NLL curve with a shallow valley near the start and a deeper one farther away (Bolker's Figure 7.1). Bolker's own story (7.4.4, p. 323-325): in a published tadpole predation analysis, Nelder-Mead found one minimum; later, BFGS from the same start found a different, slightly better one.

The idea (Bolker p. 306). Take random steps, and *sometimes accept a worse one*, so the search can climb out of a shallow valley. The name comes from metalworking: cooling metal slowly lets it settle into a low-energy state.

1. From the current point, propose a random nearby point.
2. If its NLL is lower, move there.
3. If it's higher by Δ, move there anyway with probability $e^{-\Delta/T}$, where T is the "temperature."
4. Repeat, and lower T every so often.

```
accept_tab <- outer(c(10, 1, 0.1), c(0.5, 2, 10),
                    function(temp, delta) exp(-delta / temp))
dimnames(accept_tab) <- list(paste("T =", c(10, 1, 0.1)),
                             paste("NLL worse by", c(0.5, 2, 10)))
accept_show <- ifelse(accept_tab < 0.001, "<0.001",
                      formatC(accept_tab, format = "f", digits = 3))
noquote(accept_show)
```

```
##         NLL worse by 0.5 NLL worse by 2 NLL worse by 10
## T = 10  0.951            0.819          0.368
## T = 1   0.607            0.135          <0.001
## T = 0.1 0.007            <0.001         <0.001
```

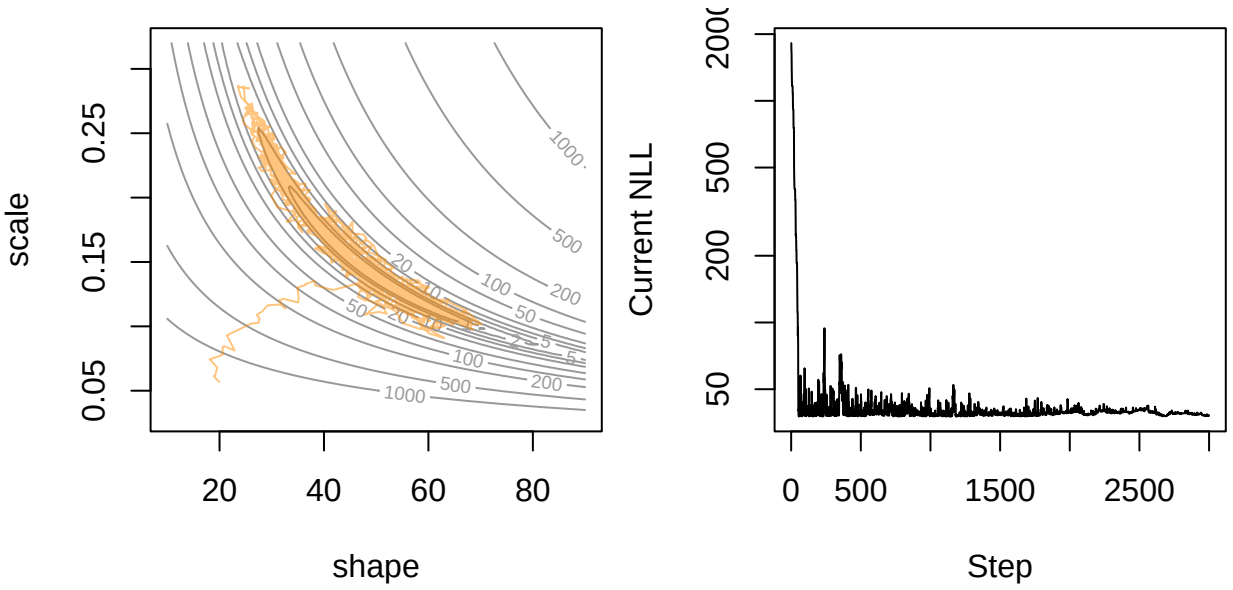
- Hot ($T = 10$): most steps are accepted; the search wanders widely. Cold ($T = 0.1$): it's almost pure downhill.

- Start hot to explore, cool down to settle.

```r
# simulated annealing: accept worse steps w.p. exp(-delta / temp); cool temp over time
anneal <- function(f, start, step_sd, n_steps = 3000, temp0 = 10,
                   cool = 0.9, every = 100) {
  current <- start
  f_current <- f(current)
  best <- current
  f_best <- f_current
  temp <- temp0
  track <- matrix(NA, n_steps, 4,
                  dimnames = list(NULL, c("shape", "scale", "nll", "temp")))
  for (i in seq_len(n_steps)) {
    proposal <- current + rnorm(length(current), 0, step_sd)
    f_prop <- f(proposal)
    if (is.finite(f_prop) && runif(1) < exp(-(f_prop - f_current) / temp)) {
      current <- proposal
      f_current <- f_prop
    }
    if (f_current < f_best) {
      best <- current
      f_best <- f_current
    }
    track[i, ] <- c(current, f_current, temp)
    if (i %% every == 0) temp <- temp * cool
  }
  list(best = best, value = f_best, track = track)
}
set.seed(746)
sa <- anneal(nll_gamma, c(20, 0.05), step_sd = c(2, 0.005))
round(c(shape = sa$best[1], scale = sa$best[2], nll = sa$value), 4)
```

```
##   shape   scale     nll
## 50.1824  0.1380 37.6680
```

```r
par(mfrow = c(1, 2), mar = c(4, 4, 0.5, 0.5))
plot_surface()
lines(sa$track[, "shape"], sa$track[, "scale"], col = adjustcolor("darkorange", 0.5))
plot(sa$track[, "nll"], type = "l", log = "y", xlab = "Step", ylab = "Current NLL")
```



- Left: the search drops into the valley within a few dozen steps, then wanders up and down it (it keeps accepting slightly worse points). Right: as T falls, the wandering settles near the minimum.
- Best point found: (50.18, 0.138), NLL 37.668, after 3,000 evaluations (Nelder-Mead: 147). Slow and imprecise on a one-valley surface; its value is on surfaces with many valleys.

Pause for discussion (~2 min).

```
set.seed(746)
fit_sann <- optim(c(20, 0.05), nll_gamma, method = "SANN")
round(c(shape = fit_sann$par[1], scale = fit_sann$par[2], nll = fit_sann$value,
        evals = unname(fit_sann$counts[1]),
        convergence = fit_sann$convergence), 4)
```

```
##       shape       scale         nll       evals convergence
##     23.1910      0.2978     40.7738  10000.0000      0.0000
```

`optim()` has a built-in simulated annealer, `method = "SANN"`. With its default settings it returns NLL 40.77 after 10,000 evaluations, and `convergence = 0`. Nelder-Mead found 37.67 in 147. Is SANN broken? Should you trust `convergence = 0`?

Where to land: not broken, but untuned. SANN runs a fixed number of steps and *always* reports `convergence = 0` (see `?optim`), so that code tells you nothing. Simulated annealing needs its temperature and step size tuned to the problem (Bolker p. 310). The usual practice: use a global method to find the right valley, then polish with Nelder-Mead or BFGS started from its answer. And whatever the method, try several starting points.

Bridge to Week 9. Keep the acceptance rule, fix $T = 1$, never cool, and keep *every* point visited instead of only the best. The collection of points then forms a sample from the

posterior distribution. That's MCMC (the Metropolis algorithm), Monday's topic.

5. Return to the big picture (~1 min)

- Point at the FIT box. A fit we can't trust can't be compared, so checking convergence comes before COMPARE.
- Fitting = searching the NLL surface. Grids are simple and hard to fool but don't scale.
- Derivative-based (Newton, BFGS): fast near the answer, can misbehave far from it. Derivative-free (Nelder-Mead): slower, robust to bumps. Stochastic (simulated annealing): slowest, can escape local valleys.
- Whatever you use: check `$convergence`, try several starts, and look at the surface (or a profile) when something seems off.
- Hand out *Curvature by Hand* (`wod/WOD_Week8_Lecture2_CurvatureByHand.Rmd`) as optional practice, not collected: it connects today's Hessian back to standard errors.

If time, or Week 9 / Lab 7

Reparameterize. Fit the Gamma in terms of its mean and shape (scale = mean/shape, Bolker's parameterization for Gamma regressions):

```
nll_mean_shape <- function(p) nll_gamma(c(p[2], p[1] / p[2]))
fit_ms <- optim(c(7, 20), nll_mean_shape)
vc_ms <- solve(optimHess(fit_ms$par, nll_mean_shape))
round(c(mean = fit_ms$par[1], shape = fit_ms$par[2], nll = fit_ms$value,
        evals = unname(fit_ms$counts[1]),
        corr = cov2cor(vc_ms)[1, 2]), 3)
```

```
##   mean  shape    nll  evals   corr
##  6.924 49.607 37.667 65.000  0.000
```

Same minimum, correlation near zero, and a round valley instead of a banana.

Checklist when fitting (Bolker 7.4). Test the NLL at the starting values (finite?). Check `$convergence`. Refit from the answer, and from a few other starts. Try a second method. Put parameters on comparable scales (log, or `control = list(parscale = ...)`). Plot a slice or profile around the answer.

PART 2: PODIUM CHEAT SHEET

Terse on purpose. Full rationale and code: Part 1.

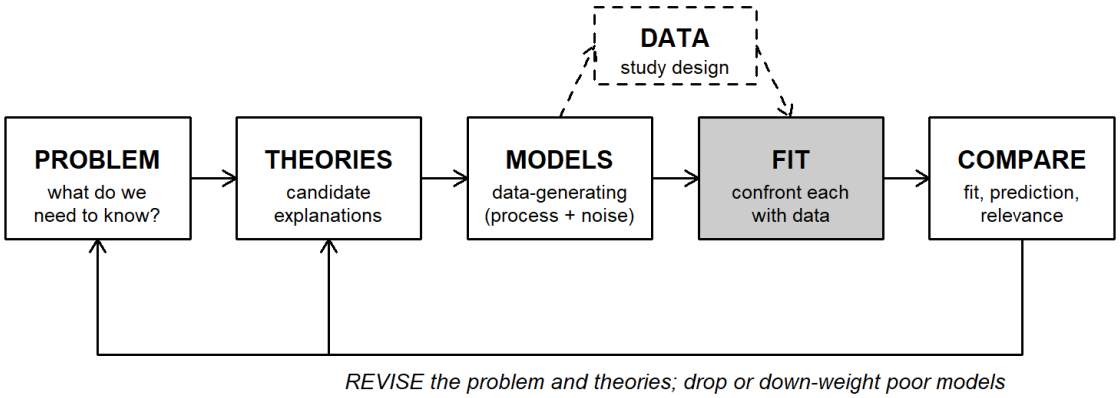
Take-home debrief ~8 min. Lecture ~40 min (3 pauses). No separate discussion.

0. Take-home debrief (~8 min)

- 3a: student puts $\lambda^* = \lambda - \text{NLL}'/\text{NLL}''$ on board (line through current slope hits 0)
- 3b: $2 \to 2.667 \to 2.963 \to 3.000$; error 1, 0.33, 0.037, 0.0005
- 3c: from $7 \to -2.33$. Why? NLL' levels off $\to$ shallow line crosses 0 far left
- Board: fixes (shorter steps, better start, bounds, log scale)
- Log scale from 7: 3.95, 3.11, 3.002
- 2a: forward slope at 2 = true slope at 2.25 (chord midpoint)
- 2b: $[f(x+\Delta) - 2f(x) + f(x-\Delta)]/\Delta^2$ = Lab 6 Hessian
- 3d: forward slope $\to$ 2.44 vs. 2.67; smaller Δ fixes it
- **Every fix you proposed is built into real optimizers**

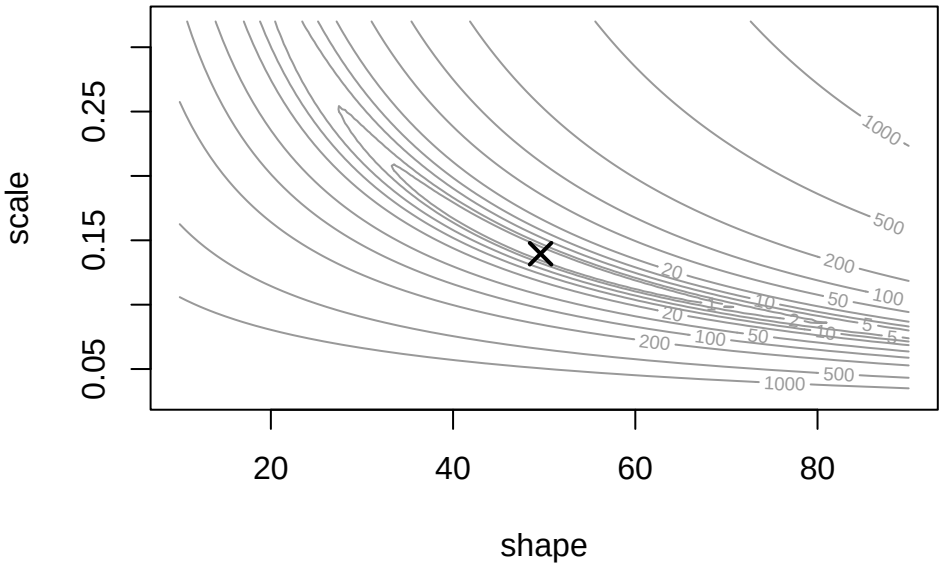
1. Big picture: new workflow (~5 min)

Draw on board in full (first time):



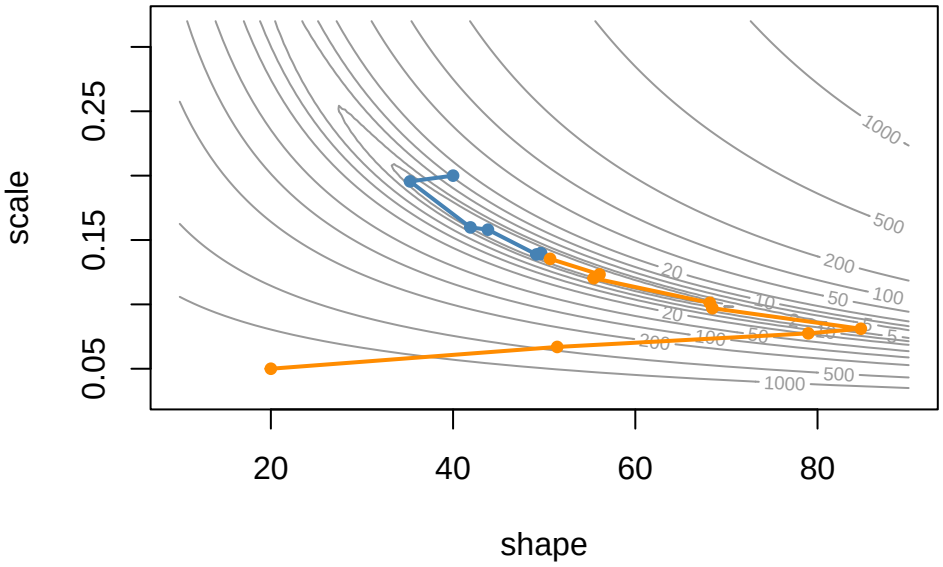
- PROBLEM first → lets us judge *relevance* later
- MODELS = process + noise; side arrow: models decide what DATA to collect
- COMPARE: fit, prediction, relevance; drop or down-weight
- Down-weight ≠ "winner is true"; survivors are corroborated
- REVISE problem + theories, not just parameters
- **Today = FIT box.** Don't trust a fit blindly before COMPARE

- Fit = find the bottom of the NLL surface
- Myxo titers, 27 rabbits, Gamma(shape, scale)
- MLE (49.60, 0.140), NLL 37.667
- Grid: 2,730 evals here; 10 params at 30 values ≈ 19,000 years



2. Newton in 2D (~10 min)

- Slope → gradient g; curvature → Hessian H
- $\theta_{\text{new}} = \theta - H^{-1}g$ (`solve(H, g)`)
- Good start: ~6 steps. Bad start: lurches, NLL goes up



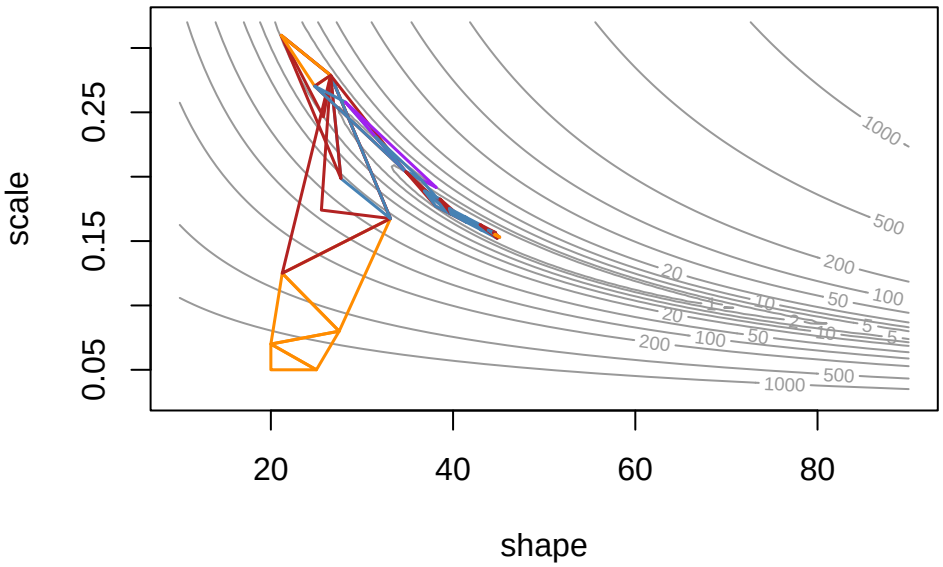
- BFGS: gradient only, builds approximate H; `mle2()` default
- 1 parameter: `optimize()`

Pause: NM 37.667 (conv 0) vs. BFGS 37.670 (conv 1). Done?

- No: hit maxit. Check `$convergence`. Log scale: 33 evals, conv 0

3. Nelder-Mead (~10 min)

- Sketch triangle: reflect, expand, contract, shrink
- No derivatives; tumbles downhill, crawls along valleys
- `optim()` default; 147 evals



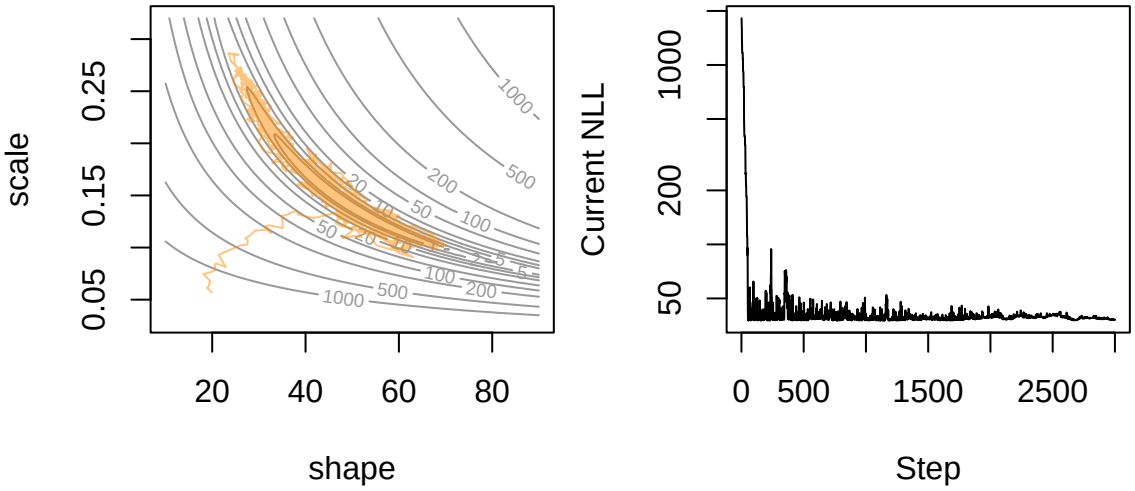
Pause: corr(shape, scale) = -0.995. What can the data tell us?

- Mean = shape × scale = 6.92: well determined; split isn't
- Lab 6 centering; reparameterize (if time)

4. Simulated annealing (~8 min; rest to Week 9)

- Board: two valleys; downhill finds the nearest one
- Bolker's tadpoles: NM and BFGS found different minima
- Random step; worse by Δ: accept w.p. $e^{-\Delta/T}$; cool T

```
##          NLL worse by 0.5 NLL worse by 2 NLL worse by 10
## T = 10   0.951            0.819          0.368
## T = 1    0.607            0.135          <0.001
## T = 0.1  0.007            <0.001         <0.001
```



Pause: SANN defaults: NLL 40.77, 10,000 evals, conv 0. Broken?

- Untuned; always reports 0. Find the valley, then polish with NM/BFGS

5. Wrap (~1 min)

- Point at FIT box: untrusted fit → can't COMPARE
- Grid / Newton-BFGS / Nelder-Mead / annealing: speed vs. robustness
- Check convergence; several starts; look at the surface
- Monday: $T = 1$, keep every point → MCMC
- **Hand out optional Curvature by Hand WOD (not collected)**